

4_gyakorlat _ feladatai_2026_Nappali tagozat

1. feladat

Adottak az A és B tömbök. Készítsük el a metszetüket és uniójukat! Írjuk ki az eredeti tömböket és az eredmény tömböket is a képernyőre!

```
int A_tomb[10]={13, 28, 43,37,10,5,8,99,101,2};
```

```
int B_tomb[5]={6,43,37,10,88};
```

//Adottak az A és B tömbök. Készítsük el a metszetüket és uniójukat!

//Írjuk ki az eredeti tömböket és az eredmény tömböket is a képernyőre!

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
// Adatbevitel
```

```
int A_tomb[10]={13, 28, 43,37,10,5,8,99,101,2};
```

```
int B_tomb[5]={6,43,37,10,88};
```

```
int i, j, cdb = 0, Metszet_tomb[15], Unio_tomb[15];
```

```
// tömbök kiírása képzés bemutatásához
```

```
for(i=0; i<10;i++)
```

```
{
```

```
    cout<<A_tomb[i]<<" ";
```

```
}
```

```
cout<<endl;
```

```
for(j=0; j<5;j++)
```

```
{
```

```
    cout<<B_tomb[j]<<" ";
```

```
}
```

```
cout<<endl<<endl;
```

```
// metszetképzés
```

```
for(i=0;i<10;i++)
```

```
{
```

```
    j=0;
```

```
    while (j < 5 && A_tomb[i] != B_tomb[j])
```

```
    {
```

```
        j++;
```

```
    }
```

```
    if(j<5)
```

```
    {
```

```
        Metszet_tomb[cdb]=A_tomb[i];
```

```
        cdb++;
```

```
    }
```

```
}
```

```
// metszet tömb kiírása
```

```
for (i = 0; i < cdb; i++)
```

```
{
```

```
    cout << Metszet_tomb[i] << " ";
```

```
}
```

```

// unióképzés
    for(i=0;i<10;i++)
    {
        Unio_tomb[i]=A_tomb[i];
    }

    cdb=10;
    for(j=0;j<5;j++)
    {
        i=0;
        while (i < 10 && A_tomb[i] != B_tomb[j])
        {
            i++;
        }
        if(i>9)
        {
            Unio_tomb[cdb]=B_tomb[j];
            cdb++;
        }
    }
}
// unio tömb kiírása
cout<< endl;
for (i = 0; i < cdb; i++)
{
    cout << Unio_tomb[i] << " ";
}
cout<<endl;
return 0;
}

```

2. feladat

Egy utazó ügynök az alábbi városokban járt januárban: Kecskemet, Szeged, Pecs, Baja, Győr, Miskolc, Vac, Cegléd, Sopron, Eger.

Februárban az alábbi városokat látogatta meg: Budapest, Kecskemet, Szeged, Pecs, Baja, Esztergom, Salgotarjan.

Azt tudjuk, hogy január hónapban 10 napot februárban 7 napot dolgozott. Készítsünk programot mely kimutatja:

1 - Mely városokban járt mindkét hónapban? (Metszet)

2 - Mely városokban volt legalább egyszer a két hónap alatt? (Unio)

A feladatot függvényekkel oldjuk meg!

```
#include <iostream>
#include <string>
using namespace std;
int Metszetkepzes(string* t1, string* t2, string* M, int a, int b);
int Uniokepzes(string* t1, string* t2, string* U, int a, int b);
int main()
{
    // Adatbevitel
    string A[] = { "Kecskemet", "Szeged", "Baja", "Győr", "Miskolc", "Vac", "Cegléd",
    "Sopron", "Eger", "Pecs" };
    string B[] = { "Budapest", "Kecskemet", "Szeged", "Pecs", "Baja", "Esztergom",
    "Salgotarjan" };
    int jdb = 10; // januári napok száma
    int fdb = 7; // februári napok száma
    string Metszet[7], Unio[17];
    cout << endl << "Az alábbi városokban járt mindkét hónapban :\n";
    int Metszdb = 0; // mindkét hónapban darabszáma
    Metszdb = Metszetkepzes(A, B, Metszet, jdb, fdb);
    for (int i = 0; i < Metszdb; i++)
    {
        cout << Metszet[i] << " ";
    }
    cout << endl << "Az alábbi városokban járt valamelyik hónapban :\n";
    int Uniodb = 0; // Egyik hónap ( unio) darabszáma
    Uniodb = Uniokepzes(A, B, Unio, jdb, fdb);
    for (int i = 0; i < Uniodb; i++)
    {
        cout << Unio[i] << " ";
    }
    return 0;
}
int Metszetkepzes(string* t1, string* t2, string* M, int a, int b)
{
    int Mdb = 0; // a metszettömb indexe
```

```

    for (int i = 0; i < a; i++)
    {
        int j = 0;
        while (j < b && t1[i] != t2[j])
            j++;
        if (j < b)
        {
            M[Mdb] = t1[i];
            Mdb++;
        }
    }
    return Mdb;
}

int Uniokepzes(string* t1, string* t2, string* U, int a, int b)
{
    int Udb = 0; //az uniotömb indexe
    for (int i = 0; i < a; i++)
    {
        U[i] = t1[i];
    }
    Udb=a;
    for (int j = 0; j < b; j++)
    {
        int i = 0;
        while (i < a && t1[i] != t2[j])
            i++;
        if (i == a)
        {
            U[Udb] = t2[j];
            Udb++;
        }
    }
    return Udb;
}

```

3. feladat

Egy 16 csapatos labdarúgó bajnokság egyik fordulójának eredményeit tároljuk a merkozes.txt állományban. Ebben a fordulóban nem biztos, hogy 8 mérkőzés volt, néhány meccs elmaradt a rossz időjárás miatt. Minden mérkőzés eredménye új sorban van és a gólok, valamint a csapatok száma szóközzel elválasztva. (3 2 ute vac)

1. Olvassa be a fájlt egy olyan adatszerkezetbe, amit az alábbi kérdésekhez fel tud használni (struktúra)) és írassa ki: Hány mérkőzés volt a fordulóban!
2. Hány csapat nyert otthon?
3. Melyik csapat rúgta a legtöbb gólt?
4. Volt -e döntetlen?
5. Hány gólt lőtt az ute?(Nem tudjuk, hogy egyáltalán játszott-e)

AZ EGYES RÉSZFELADATOKAT FÜGGVÉNYEKKELD OLDJUK MEG!!!

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct fordulo
{
    int lott, kapott;
    string nev1, nev2;
};
int Beolvas(fordulo* tmb, string nev);
int Hazai_nyert(fordulo* tmb, int n);
string Legtobb_gol(fordulo* tmb, int n);
bool Dontetlen(fordulo* tmb, int n);
int UTE_gol(fordulo* tmb, int n, string nev);
int main()
{
    fordulo A[8];
    string fnev = "merkozes.txt";
    int db = Beolvas(A,fnev);
    cout << "A forduloban: " << db << " merkozest jatszottak" << endl;
    cout<<Hazai_nyert(A,db)<<" Csapat gyozott otthon"<<endl;
    cout << "A legtoobb golt a " << Legtobb_gol(A,db) << " lotte" << endl;
    if(Dontetlen(A,db)) cout<<"Volt dontetlen merkozes" << endl;
        else cout<< "Nem volt dontetlen merkozes" << endl;
    int utegol = UTE_gol(A, db, "ute");
    if (utegol > -1) cout << utegol << " golt lott az ute" << endl;
    else cout<<"Nem jatszott az UTE" << endl;
    return 0;
}
// Adatbevitel
int Beolvas(fordulo* tmb, string nev)
{
    ifstream be(nev);
    if (be.fail()) { cout << "hiba"; system("pause"); exit(1); }
    int i = 0, j, db = 0;
    while (!be.eof())
    {
```

```

        be >> tmb[i].lott >> tmb[i].kapott >> tmb[i].nev1 >> tmb[i].nev2;
        i++;
    }
    be.close();
    return i;
}
// ? csapat nyert otthon.
int Hazai_nyert(fordulo* tmb, int n)
{
    int nyert = 0;
    for (int i = 0; i < n; i++)
    {
        if (tmb[i].lott > tmb[i].kapott)
            nyert++;
    }
    return nyert;
}
// ? Melyik csapat rúgta a legtöbb gólt?
string Legtobb_gol(fordulo* tmb, int n)
{
    string maxcsapat = tmb[0].nev1; // most nem a sorszámot jegyezzük meg, mert
    minden sorban(struktúra adatban) két adat -hazai, vendég van
    int max = -1;                // a legtöbb gól száma
    for (int i = 0; i < n; i++)
    {
        if (tmb[i].lott > max) { max = tmb[i].lott; maxcsapat = tmb[i].nev1; }
        if (tmb[i].kapott > max) { max = tmb[i].kapott; maxcsapat = tmb[i].nev2; }
    }
    return maxcsapat;
}
// Volt-e döntetlen?
bool Dontetlen(fordulo* tmb, int n)
{
    int i = 0;
    while (i < n && tmb[i].lott != tmb[i].kapott)
        i++;
    if (i < n) return true;
    else return false;
}
// ? golt lott az ute -- nem tudjuk, hogy jatszott-e
int UTE_gol(fordulo* tmb, int n, string nev)
{
    int i = 0;
    while (i < n && !(tmb[i].nev1 == nev || tmb[i].nev2 == nev))
        i++;
    if (i == n) return -1;
    else
        if (tmb[i].nev1 == nev) return tmb[i].lott;
        else return tmb[i].kapott;
}

```

4. feladat – a zh is hasonló lesz

Egy 7 fős kézilabdacsapatról egy mérkőzésen statisztika készült. Tárolták a játékosok nevét, játékidjét, és dobott góljait egy jegyzokonyv.txt nevű állományban. Sajnos a jegyzőkönyv megsérült és csak a játékosok neve maradt meg az állományban. Állítsuk vissza az adatokat úgy, hogy a neveket a fájlból, a játékidőt (0-60) és a dobott gólokat (0-12) véletlen generátorral állítsuk elő!

Feladatok:

1. Hozzunk létre egy megfelelő adatszerkezetet az adatok tárolására!
2. Töltsük fel a struktúratömböt adatokkal úgy, hogy a nevet a fájlból olvassuk be, a játékidőt (0-60) és a dobott gólokat (0-12) véletlen generátorral állítsuk elő!
3. Írassuk ki az adatokat táblázatosan!
4. Számítsuk ki hány gólt dobott a csapat összesen!
5. Ki dobta a legtöbb gólt (adjuk meg a nevét és góljainak számát)?
6. Számoljuk meg hányan dobtak 6 gól felett?
7. Írassuk ki azoknak a nevét és játékidjét, akik legalább 20 percet a pályán töltöttek
8. A kiegészített adatbázist, név, játékidő, gólok mentjük el az uj_jegyzokonyv.txt állományba!

```
#include<iostream>
#include<string>
#include<ctime>
#include<iomanip>
#include<fstream>
using namespace std;
struct jatekos
{
    string nev;
    int jatekido;
    int gol;
};
int main()
{
    int letszam;
    //1,Struktúra tömb létrehozása az adatok tárolásához - tudjuk, hogy 7 fő játszott
    jatekos csapat[7];
    //2, Csapat adatainak bekérése
    ifstream be("jegyzokonyv.txt");
    srand((unsigned int)time(NULL));
    for (int i = 0; i < 7; i++)
    {
        getline(be, csapat[i].nev);
        csapat[i].jatekido = rand() % 61;
        csapat[i].gol=rand()%11;
    }
    //3, Csapattagok adatainak táblázatos kiírása // formázott kiírás
    cout.setf(ios::left);
    for (int i = 0; i < 7; i++)
    {
        cout << setw(20) << csapat[i].nev << setw(2) << csapat[i].jatekido << " " <<
        csapat[i].gol << endl;
    }
}
```

```

//4, Számítsuk ki a csapat össz. dobott góljának számát!
    int ossz = 0;
    for (int i = 0; i < 7; i++)
    {
        ossz = ossz + csapat[i].gol;
    }
    cout << "A csapat osszgolszama: " << ossz << endl;
//5, Legtobb golt dobo jatekos neve, dobott goljainak szama
    int max = 0;
    for (int i = 1; i < 7; i++)
    {
        if (csapat[i].gol > csapat[max].gol) { max = i; }
    }
    cout << "A legtobb golt dobo jatekos: " << csapat[max].nev << " goljainak szama: "
<< csapat[max].gol << endl;
//6, A 6 gól felett dobók száma
    int db6 = 0;
    for (int i = 0; i < 7; i++)
    {
        if (csapat[i].gol > 6)
        {
            db6++;
        }
    }
    cout << "A hat gol felett dobok szama: " << db6 << endl;
// 7, Írassuk ki azoknak a nevét és játékidjét akik legalább 20 percet a pályán töltöttek
    cout << "A legalabb 20 percet játszo jatekosok: " << endl;
    for (int i = 0; i < 7; i++)
    {
        if (csapat[i].jatekido >= 20)
        {
            cout << setw(20) << csapat[i].nev << " " << csapat[i].jatekido <<
endl;
        }
    }
//8,A kiegészített adatbázist, név, játékidő, gólok mentsük el az uj_jegyzokonyv.txt
állományba!
    ofstream ki("uj_jegyzokonyv.txt");
    ki.setf(ios::left);
    for (int i = 0; i < 7; i++)
    {
        ki<< setw(20) << csapat[i].nev << setw(2) << csapat[i].jatekido << " " <<
csapat[i].gol << endl;
    }
    return 0;
}

```

Házi Feladat:

KÉSZÍTSE EL A FELADATOT ÚGY IS, HOGY AZ EGYES RÉSZFELADATOKAT FÜGGVÉNYEKKELD OLDJA MEG!